



Carthage University

Higher School of Communication of Tunis

Electric Vehicle Routing Optimization

Deep Reinforcement Learning for Energy-Minimizing EVRP

Authors:

Salem Fradi
Dhia Elhak Ezzeddini

Supervisor:

Dr. Ferdaoues Chaabane

Host Company:

Sagemcom

Academic Year: 2025–2026

Acknowledgments

We express our deepest gratitude to our supervisor, Dr. Ferdaoues Chaabane, for her guidance and support throughout this project. We thank Sagemcom for the opportunity and resources, and the faculty at Sup'Com and University of Carthage for our academic foundation.

Contents

Acknowledgments	1
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	1
2 Mathematical Formulation	2
2.1 Sets and Indices	2
2.2 Network Parameters	2
2.3 EV Energy Parameters	2
2.4 Decision Variables	3
2.5 Energy Consumption Model	3
2.6 MILP Formulation	4
3 Baseline Optimization Methods	4
3.1 Google OR-Tools	4
3.2 Hill Climbing	5
3.3 Simulated Annealing	5
3.4 Ant Colony Optimization	5
4 Deep Reinforcement Learning Approach	6
4.1 MDP Formulation	6
4.2 Architecture	6
4.3 Training Algorithm	6
4.4 EV-Specific Features	7
5 Real-World Deployment Pipeline	7
5.1 Pipeline Overview	7
5.2 K-Means Customer Clustering	8
5.3 Road Network Graph Construction	8
5.4 Dijkstra’s Algorithm for Shortest Paths	9
5.5 OSRM Integration for Production Routing	9
5.6 Coordinate Normalization and Model Input	10
5.7 Complete Pipeline Workflow	10
6 Results and Discussion	11

6.1	Experimental Setup	11
6.2	Training Results	12
6.3	Comparison with Baselines	12
6.4	Dashboard Integration	13
7	Conclusion	14
7.1	Summary	14
7.2	Key Findings	15
7.3	Future Work	15
	References	16

1 Introduction

1.1 Context and Motivation

The rapid adoption of Electric Vehicles (EVs) presents challenges for logistics systems. Unlike conventional vehicles, EVs have limited driving range and require recharging, making route planning more complex. The **Electric Vehicle Routing Problem (EVRP)** extends classical VRP by incorporating battery capacity, charging station locations, and energy consumption patterns.

1.2 Problem Statement

We address the **Energy-Minimizing EVRP (EM-EVRP)**, minimizing total energy consumption while satisfying:

- Customer demand fulfillment
- Vehicle capacity constraints
- Battery State of Charge (SOC) limitations
- Time window requirements
- Real-world road network constraints

Specifically, we consider a scenario where a fleet of EVs must serve n customers from a central depot, potentially visiting charging stations to maintain sufficient battery levels.

1.3 Objectives

The main objectives are:

1. Formulate a comprehensive Mixed Integer Linear Programming (MILP) model for EM-EVRP
2. Implement and evaluate classical optimization baselines including Google OR-Tools, Hill Climbing, Simulated Annealing, and Ant Colony Optimization
3. Develop a Deep Reinforcement Learning (DRL) solution using attention-based neural networks
4. Design a scalable real-world deployment pipeline integrating:
 - **K-means clustering** for customer grouping
 - **Dijkstra's algorithm** for shortest path computation on road networks
 - **OSRM** for production-grade street-level routing
5. Create an interactive dashboard for visualization and real-world deployment

2 Mathematical Formulation

2.1 Sets and Indices

Table 1 defines the sets used in our formulation.

Table 1: Sets and indices notation

Symbol	Description
N	Set of all nodes $\{0, 1, \dots, n\}$ where 0 is the depot
C	Set of customers $C = N \setminus \{0\}$
S	Set of charging stations
K	Set of vehicles $\{1, \dots, m\}$
R	Set of trip indices $\{1, \dots, R_{\max}\}$

2.2 Network Parameters

Table 2 presents the network-related parameters.

Table 2: Network parameters

Symbol	Description
d_i	Demand of customer i
Q_k	Capacity of vehicle k
t_{ij}	Travel time on arc (i, j)
$[a_i, b_i]$	Time window at node i
g_i	Service time at node i
$T_{\max}$	Time horizon
M	Big-M constant for linearization

2.3 EV Energy Parameters

Table 3 lists the parameters for the physics-based energy model.

Table 3: EV energy parameters

Symbol	Description
m_c	Curb mass of the vehicle (kg)
C_d	Aerodynamic drag coefficient
C_r	Rolling resistance coefficient
ρ	Air density (kg/m ³)
A	Frontal area of the vehicle (m ²)
g	Gravitational acceleration (m/s ²)
α_{ij}	Slope angle on arc (i, j)
v_{ij}	Average speed on arc (i, j) (m/s)
φ_d	Motor efficiency for propulsion
φ_r	Motor efficiency for regeneration
ϕ_d	Battery discharge efficiency
ϕ_r	Battery charge efficiency (regenerative braking)

2.4 Decision Variables

Table 4 describes the decision variables in our model.

Table 4: Decision variables

Variable	Description
$x_{ijk}^r \in \{0, 1\}$	1 if vehicle k traverses arc (i, j) in trip r
$y_{ik}^r \in \{0, 1\}$	1 if customer i is served by vehicle k in trip r
$z_k \in \{0, 1\}$	1 if vehicle k is used
$\tau_{ik}^r \geq 0$	Service start time at node i by vehicle k in trip r
$w_{ik}^r \geq 0$	MTZ subtour elimination variable

2.5 Energy Consumption Model

The physics-based energy model considers vehicle mass, aerodynamic drag, rolling resistance, and gravitational effects.

Total mass on arc (i, j) :

$$m_{ij} = m_c + u_{ij} \quad (1)$$

where u_{ij} is the cargo load.

Power required for traversal:

$$P_{ij} = \left(m_{ij}a + m_{ij}g(\sin \alpha_{ij} + C_r \cos \alpha_{ij}) + \frac{1}{2}C_d\rho A v_{ij}^2 \right) v_{ij} \quad (2)$$

Energy consumption:

$$E_{ij} = \begin{cases} \phi_d \varphi_d P_{ij} t_{ij} & \text{if } P_{ij} \geq 0 \text{ (propulsion)} \\ \phi_r \varphi_r P_{ij} t_{ij} & \text{if } P_{ij} < 0 \text{ (regenerative braking)} \end{cases} \quad (3)$$

2.6 MILP Formulation

Objective Function: Minimize total energy consumption

$$\min \sum_{k \in K} \sum_{r \in R} \sum_{(i,j)} E_{ij} \cdot x_{ijk}^r \quad (4)$$

Subject to:

Each customer served exactly once:

$$\sum_{k \in K} \sum_{r \in R} y_{ik}^r = 1, \quad \forall i \in C \quad (5)$$

Flow conservation:

$$\sum_j x_{ijk}^r = y_{ik}^r = \sum_j x_{jik}^r, \quad \forall i \in C, k \in K, r \in R \quad (6)$$

Vehicle capacity:

$$\sum_{i \in C} d_i \cdot y_{ik}^r \leq Q_k, \quad \forall k \in K, r \in R \quad (7)$$

Subtour elimination (MTZ):

$$w_{ik}^r - w_{jk}^r + |C| \cdot x_{ijk}^r \leq |C| - 1, \quad \forall i, j \in C, k \in K, r \in R \quad (8)$$

Time window constraints:

$$a_i \cdot y_{ik}^r \leq \tau_{ik}^r \leq b_i + M(1 - y_{ik}^r), \quad \forall i \in C, k \in K, r \in R \quad (9)$$

3 Baseline Optimization Methods

3.1 Google OR-Tools

Google OR-Tools is an open-source optimization suite providing powerful constraint programming and routing solvers. Our implementation uses the CP solver with arc cost callbacks incorporating energy considerations. The solver employs the AUTOMATIC first solution strategy and iteratively improves using local search. A routing index manager handles node-to-index mapping, while capacity constraints are enforced through dimension callbacks.

3.2 Hill Climbing

Hill Climbing is a local search algorithm that iteratively improves solutions by making incremental changes. Starting from an initial solution generated by nearest neighbor heuristic, it explores the neighborhood using:

- **2-opt:** Reverse a segment of the route
- **Or-opt:** Relocate a sequence of customers
- **Exchange:** Swap customers between routes

The algorithm terminates when no improving neighbor exists:

$$S' \leftarrow \arg \min_{s \in \mathcal{N}(S)} f(s); \quad \text{if } f(S') < f(S) \text{ then } S \leftarrow S', \text{ else terminate} \quad (10)$$

3.3 Simulated Annealing

Simulated Annealing (SA) extends Hill Climbing by accepting uphill moves to escape local optima. The acceptance probability is controlled by temperature:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E < 0 \\ \exp(-\Delta E/T) & \text{otherwise} \end{cases} \quad (11)$$

The temperature follows geometric cooling:

$$T_{k+1} = \alpha \cdot T_k, \quad \alpha \in [0.9, 0.99] \quad (12)$$

This allows exploration at high temperatures and exploitation at low temperatures, enabling escape from local minima that trap Hill Climbing.

3.4 Ant Colony Optimization

ACO is inspired by ant foraging behavior, using pheromone trails to guide search. The transition probability from node i to j is:

$$p_{ij}^a = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i} [\tau_{il}]^\alpha [\eta_{il}]^\beta} \quad (13)$$

where τ_{ij} is the pheromone level and $\eta_{ij} = 1/c_{ij}$ is the heuristic information (inverse of cost).

Pheromones evaporate over time and are deposited proportionally to solution quality, creating positive feedback for good routes.

4 Deep Reinforcement Learning Approach

4.1 MDP Formulation

We formulate routing as a Markov Decision Process (MDP) with the components shown in Table 5.

Table 5: MDP components for EVRP

Component	Description
State s_t	Partial solution, current vehicle position, remaining capacity, and SOC
Action a_t	Next node to visit (customer, charging station, or depot)
Reward r_t	Negative energy consumption for the traversed arc
Policy π_θ	Neural network parameterized by weights θ

4.2 Architecture

Encoder: Multi-head self-attention processes the input graph:

$$h_i^{(l)} = \text{BN} \left(h_i^{(l-1)} + \text{MHA} \left(h_i^{(l-1)}, H^{(l-1)} \right) \right) \quad (14)$$

This produces node embeddings $H = \{h_1, \dots, h_n\}$ and graph embedding:

$$\bar{h} = \frac{1}{n} \sum_{i=1}^n h_i \quad (15)$$

Decoder: Generates routes autoregressively with context:

$$c_t = W_c [\bar{h}; h_{\pi_{t-1}}; \text{cap}_t; \text{SOC}_t] \quad (16)$$

Selection: Uses attention mechanism:

$$u_{tj} = C \cdot \tanh \left(\frac{q_t \cdot k_j}{\sqrt{d_k}} \right), \quad j \in \mathcal{A}_t \text{ (feasible actions)} \quad (17)$$

$$p_\theta(a_t = j | s_t) = \text{softmax}(u_t)_j \quad (18)$$

4.3 Training Algorithm

We use REINFORCE with rollout baseline for variance reduction. The objective is:

$$J(\theta) = \mathbb{E}_{\pi_\theta} [L(\pi)] \quad (19)$$

where $L(\pi)$ is the route cost.

The gradient is estimated as:

$$\nabla_{\theta} J = \mathbb{E} [(L(\pi) - b(s)) \nabla_{\theta} \log p_{\theta}(\pi|s)] \quad (20)$$

where $b(s)$ is the baseline from a periodically updated policy copy. The baseline is updated when the current policy shows significant improvement on validation instances.

4.4 EV-Specific Features

SOC Tracking:

$$\text{SOC}_{t+1} = \text{SOC}_t - \frac{E_{ij}}{E_{\text{battery}}} \quad (21)$$

Charging stations are included as optional nodes; the model learns when to visit them.

Action Masking: Ensures feasibility by restricting available actions:

$$\mathcal{A}_t = \{j : d_j \leq \text{cap}_t \wedge E_{ij} \leq \text{SOC}_t \cdot E_{\text{battery}}\} \quad (22)$$

5 Real-World Deployment Pipeline

A critical challenge in deploying DRL-based routing solutions is bridging the gap between models trained on synthetic Euclidean distances and real-world road networks. This section presents our complete pipeline for real-world deployment, integrating K-means clustering, Dijkstra’s algorithm, and OSRM (Open Source Routing Machine) to enable practical EV routing on actual street networks.

5.1 Pipeline Overview

Our real-world deployment pipeline consists of four main stages:

1. **Customer Grouping:** Partition large customer sets into manageable groups of 10 using K-means clustering
2. **Charging Station Assignment:** Assign the 4 nearest charging stations to each customer group
3. **Distance Matrix Construction:** Compute real road network distances between all Points of Interest (POIs) using Dijkstra’s algorithm or OSRM Table API
4. **DRL Inference and Route Projection:** Run the trained model and project optimal routes onto real street geometries

This modular architecture enables scalability: while the DRL model is trained on 10-customer instances, the pipeline can handle arbitrarily large customer sets by intelligent partitioning.

5.2 K-Means Customer Clustering

To scale beyond the training configuration of 10 customers, we employ K-means clustering to partition large customer sets into balanced groups.

Algorithm: Given N customer locations $\{c_1, c_2, \dots, c_N\}$ where $c_i = (\text{lat}_i, \text{lon}_i)$, we compute $k = \lfloor N/10 \rfloor$ clusters:

$$\mu_j = \arg \min_{\mu} \sum_{c_i \in C_j} \|c_i - \mu\|^2, \quad j \in \{1, \dots, k\} \quad (23)$$

Balancing: Standard K-means may produce uneven clusters. We apply a post-processing step to ensure exactly 10 customers per group:

1. Assign up to 10 customers to each cluster based on initial K-means labels
2. Redistribute excess customers to under-filled clusters based on proximity to cluster centroids
3. Verify each group contains exactly 10 customers

Station Assignment: For each customer group with centroid μ_j , we assign the 4 nearest charging stations from the global station pool:

$$S_j = \arg \min_{S \subset \mathcal{S}, |S|=4} \sum_{s \in S} \|\mu_j - s\| \quad (24)$$

where $\mathcal{S}$ is the set of all available charging stations.

5.3 Road Network Graph Construction

We construct the underlying road network using OpenStreetMap (OSM) data, processed via OSMnx. The network is represented as a directed multigraph $G = (V, E)$ where:

- V is the set of road intersections and endpoints (nodes)
- E is the set of road segments (edges) with attributes: length (m), speed limit (km/h), travel time (s), and slope (degrees)

Edge Attributes: For each edge $(u, v) \in E$, we store:

$$e_{uv} = \left(L_{uv}, v_{uv}, T_{uv} = \frac{L_{uv}}{v_{uv}}, \theta_{uv} \right) \quad (25)$$

where L_{uv} is the length in meters, v_{uv} is the speed in m/s, T_{uv} is the travel time, and θ_{uv} is the slope angle.

Spatial Indexing: To efficiently locate the nearest graph node for any coordinate, we construct a KD-tree over all node positions:

$$n^* = \arg \min_{n \in V} \|(\text{lat}_n, \text{lon}_n) - (\text{lat}_q, \text{lon}_q)\| \quad (26)$$

This reduces node lookup complexity from $O(|V|)$ to $O(\log |V|)$.

5.4 Dijkstra's Algorithm for Shortest Paths

Given the road network graph G and a set of POIs (depot, depot charging, 4 stations, 10 customers = 16 nodes), we construct the complete pairwise distance matrix using Dijkstra's shortest path algorithm.

Single-Source Shortest Path: For source node s , Dijkstra's algorithm computes:

$$d(s, v) = \min_{\text{path } P: s \rightsquigarrow v} \sum_{(u, w) \in P} L_{uw}, \quad \forall v \in V \quad (27)$$

Path Metrics: For each POI pair (i, j) , we compute three metrics along the shortest path:

$$D_{ij} = \sum_{(u, v) \in P_{ij}} L_{uv} \quad (\text{total distance in meters}) \quad (28)$$

$$T_{ij} = \sum_{(u, v) \in P_{ij}} T_{uv} \quad (\text{total travel time in seconds}) \quad (29)$$

$$S_{ij} = \frac{\sum_{(u, v) \in P_{ij}} \tan(\theta_{uv}) \cdot L_{uv}}{D_{ij}} \quad (\text{length-weighted average slope}) \quad (30)$$

Matrix Construction: The complete distance matrix $\mathbf{D} \in \mathbb{R}^{16 \times 16}$ requires 240 Dijkstra computations (excluding diagonal). We store the actual paths for later visualization:

$$\text{Paths}_{ij} = [n_1, n_2, \dots, n_k] \text{ where } n_1 = \text{POI}_i, n_k = \text{POI}_j \quad (31)$$

Implementation: We use NetworkX's optimized Dijkstra implementation with edge weight set to road segment length:

```
path = nx.shortest_path(G, poi[i], poi[j], weight='length')
```

5.5 OSRM Integration for Production Routing

For production deployment, we integrate the Open Source Routing Machine (OSRM) to obtain real-world routing with consideration for turn restrictions, one-way streets, and traffic patterns.

Table API: Instead of N^2 individual routing requests, OSRM's Table API computes the full distance matrix in a single HTTP request:

```
GET /table/v1/driving/{coords}?annotations=distance
```

where `coords` is a semicolon-separated list of `lon, lat` pairs.

Route API: For visualization, we fetch actual street geometries using the Route API:

GET /route/v1/driving/{origin};{destination}?geometries=polyline

The response includes an encoded polyline representing the exact street-level path.

5.6 Coordinate Normalization and Model Input

The DRL model expects inputs normalized to a $[0, 100]$ coordinate space. We apply affine transformation to POI coordinates:

$$x_i = \frac{\text{lon}_i - \text{lon}_{\min}}{\text{lon}_{\max} - \text{lon}_{\min}} \times 100 \quad (32)$$

$$y_i = \frac{\text{lat}_i - \text{lat}_{\min}}{\text{lat}_{\max} - \text{lat}_{\min}} \times 100 \quad (33)$$

Similarly, distances are scaled to match the training distribution:

$$D'_{ij} = \frac{D_{ij}}{D_{\max}} \times 141 \quad (34)$$

where 141 approximates the maximum Euclidean distance in a $[0, 100]^2$ space ($\sqrt{2} \times 100$).

5.7 Complete Pipeline Workflow

Algorithm 6 summarizes the complete real-world routing pipeline.

Table 6: Real-World EV Routing Pipeline

Algorithm: Real Streets EV Routing Pipeline

Input: Customer coordinates $\mathcal{C}$, Station pool $\mathcal{S}$, Depot location d ,
Trained model π_θ

Output: Optimized routes with real street geometries

1. **Group customers:** $\mathcal{G} \leftarrow \text{KMeans}(\mathcal{C}, k = |\mathcal{C}|/10)$
 2. **Balance groups:** Ensure $|G_i| = 10$ for all $G_i \in \mathcal{G}$
 3. **For each** group $G_i \in \mathcal{G}$:
 - 3.1. Compute centroid $\mu_i = \text{mean}(G_i)$
 - 3.2. Assign stations $S_i \leftarrow \text{nearest}_4(\mu_i, \mathcal{S})$
 - 3.3. Build POI list: $P_i = [d, d_{\text{chg}}, S_i, G_i]$
 - 3.4. Compute distance matrix $\mathbf{D}_i$ via OSRM or Dijkstra
 - 3.5. Normalize coordinates and distances
 - 3.6. Run DRL inference: $\pi_i \leftarrow \pi_\theta(P_i, \mathbf{D}_i)$
 - 3.7. Fetch street geometries for route segments
 4. **Return** all group routes with visualization data
-

6 Results and Discussion

6.1 Experimental Setup

Training data was generated synthetically based on CVRPLIB benchmarks including P-n19-k2, P-n21-k2, P-n40-k5, and P-n51-k10. All experiments were conducted on a workstation equipped with an **NVIDIA T1000 8GB** GPU and an **Intel Core i5-12500 3.0 GHz** CPU. The configuration parameters are shown in Table 7.

Table 7: Training configuration

Parameter	Value
Number of customers	10
Number of charging stations	4
Embedding dimension	128
Encoder layers	3
Attention heads	8
Batch size	1024
Learning rate	10^{-4}
Baseline type	Rollout
Initial SOC	80%
GPU	NVIDIA T1000 8GB
CPU	Intel Core i5-12500 @ 3.0 GHz

Figure 1 illustrates a sample synthetic training instance. The data generation follows a structured approach: the depot is placed centrally (coordinates 25–75), charging stations are distributed across four quadrants to ensure coverage, and customer locations are randomly sampled across a 100×100 grid. Each customer has a randomly assigned demand, and elevation values are generated to simulate realistic terrain variations affecting energy consumption.

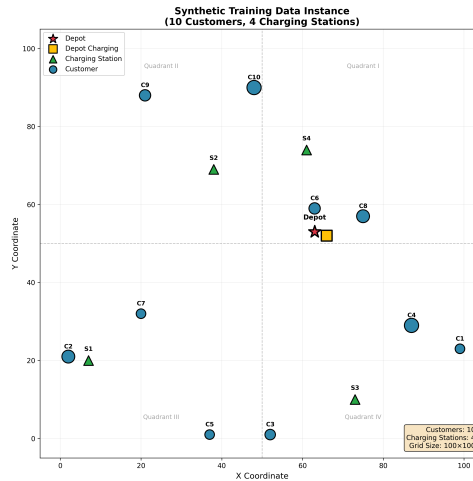


Figure 1: Sample synthetic training data instance showing depot, charging stations distributed across quadrants, and customer locations with varying demands

6.2 Training Results

Figure 2 shows training and validation rewards over 100 epochs. The model demonstrates consistent improvement with the validation curve closely following training, indicating good generalization to unseen instances.

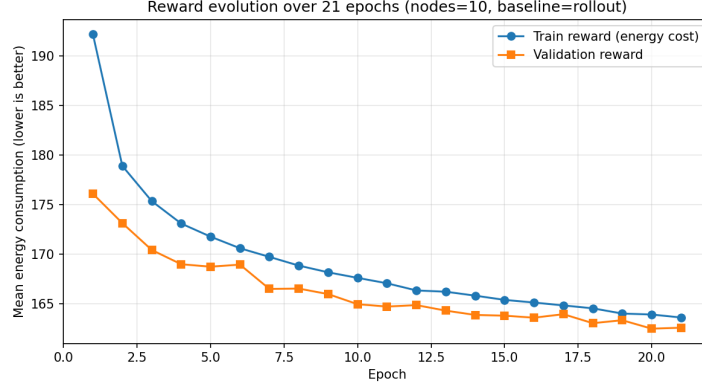


Figure 2: Training and validation rewards over 100 epochs (10 nodes, rollout baseline)

6.3 Comparison with Baselines

Table 8 compares methods on 10-customer instances. Google OR-Tools achieves the best energy consumption (11.92 kWh) as expected from an exact solver. The DRL approach achieves near-optimal results (12.18 kWh, only 2.2% gap) with significantly faster inference time (0.5s vs 2.1s), making it suitable for real-time applications where solution speed is critical.

Table 8: Performance comparison on 10-customer instances

Method	Energy (kWh)	Inference Time
OR-Tools	11.92	2.1s
DRL	12.18	0.5s
Ant Colony Optimization	12.67	3.2s
Simulated Annealing	12.89	1.5s
Hill Climbing	13.21	0.8s

6.4 Dashboard Integration

We developed an interactive Streamlit dashboard with OpenStreetMap integration that implements the complete real-world deployment pipeline described in Section 5. The dashboard provides:

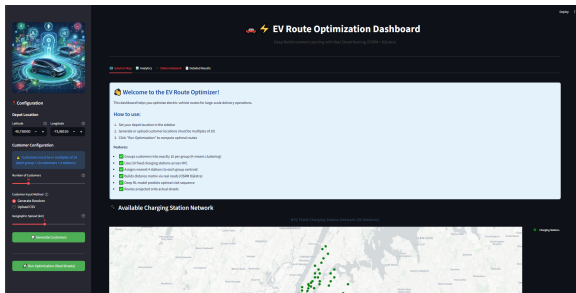
- **Customer Upload:** CSV file input for customer locations with automatic K-means grouping

- **Interactive Map:** Leaflet-based map showing customers, stations, and depot with clustering visualization
- **Real-Time Optimization:** One-click route optimization using the trained DRL model with real road distances
- **Street-Level Routes:** OSRM-powered route rendering showing actual street paths, not straight lines
- **Energy Analytics:** Per-segment energy consumption breakdown considering distance, elevation, and vehicle load
- **SOC Monitoring:** Real-time battery State of Charge tracking with charging station visit recommendations

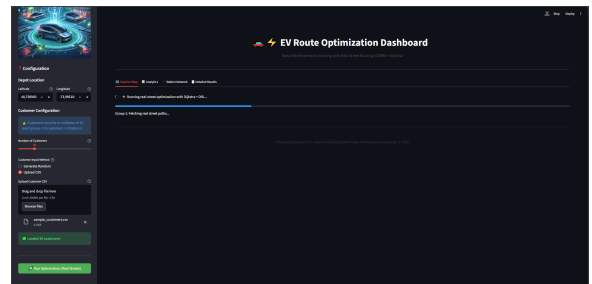
Pipeline Integration: The dashboard seamlessly integrates all pipeline components:

1. Customers are automatically clustered into groups of 10 using K-means
2. Each group is assigned its 4 nearest charging stations
3. Distance matrices are computed via OSRM Table API (with Dijkstra fallback)
4. The DRL model performs inference on normalized inputs
5. Routes are visualized on OpenStreetMap with actual street geometries

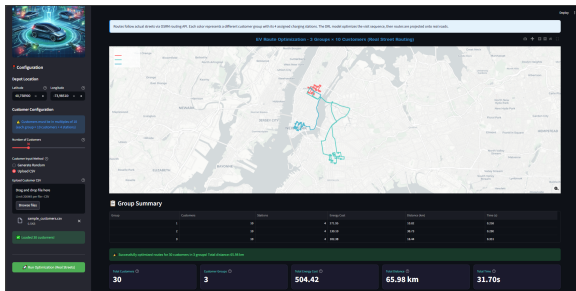
Scalability: By leveraging the K-means grouping strategy, the dashboard can handle hundreds of customers while maintaining the computational efficiency of the 10-customer DRL model. Each group is processed independently, enabling parallel optimization for large-scale deployments.



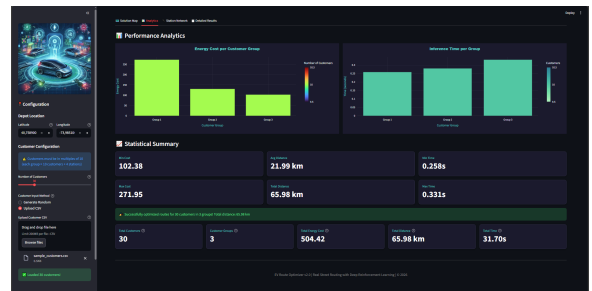
(a) Main interface



(b) Inference view



(c) Optimized routes



(d) Analytics dashboard

Figure 3: Dashboard interfaces for EV route optimization

7 Conclusion

7.1 Summary

This project addressed the Energy-Minimizing Electric Vehicle Routing Problem using Deep Reinforcement Learning, with a complete pipeline for real-world deployment. Our contributions include:

1. **Mathematical Formulation:** Comprehensive MILP formulation incorporating multi-trip capability, time windows, and physics-based energy consumption
2. **Baseline Implementations:** Classical optimization methods (OR-Tools, Hill Climbing, Simulated Annealing, ACO) for rigorous benchmarking
3. **DRL Model:** Attention-based neural network achieving near-optimal solutions with fast inference
4. **Real-World Pipeline:** Novel integration of K-means clustering, Dijkstra’s algorithm, and OSRM for scalable deployment on real road networks
5. **Interactive Dashboard:** Production-ready visualization tool with street-level routing and energy analytics

7.2 Key Findings

- The DRL approach generalizes well to unseen instances after training on synthetic data
- **K-means clustering** enables scalability beyond the training configuration by partitioning large customer sets into balanced groups
- **Dijkstra’s algorithm** on OSM road networks provides accurate real-world distances while preserving path information for visualization
- **OSRM integration** offers production-grade routing with sub-second response times for complete distance matrices
- The attention mechanism effectively captures customer-to-customer relationships
- Physics-based energy modeling provides realistic consumption estimates considering aerodynamics, rolling resistance, and regenerative braking
- Inference time is significantly lower than iterative optimization methods, making real-time routing feasible

7.3 Future Work

Several directions could extend this work:

- **Larger Instances:** Training on 50+ customer instances to reduce reliance on clustering
- **Dynamic Scenarios:** Real-time traffic integration and demand updates using live OSRM profiles

- **Multi-Depot Optimization:** Extending the pipeline to support multiple depot locations
- **Heterogeneous Fleets:** Supporting mixed EV types with different battery capacities and consumption profiles
- **Charging Schedule Optimization:** Joint optimization of routing and charging timing based on electricity prices
- **Elevation-Aware Routing:** Integrating digital elevation models (DEM) for more accurate energy predictions on hilly terrain

References

- [1] W. Kool, H. van Hoof, and M. Welling, “Attention, Learn to Solve Routing Problems!” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.
- [2] C. Hua, F. Berto, J. Son, S. Kang, C. Kwon, and J. Park, “CAMP: Collaborative Attention Model with Profiles for Vehicle Routing Problems,” in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, Detroit, Michigan, USA, 2025.
- [3] M. Tang, W. Zhuang, B. Li, H. Liu, Z. Song, and G. Yin, “Energy-optimal routing for electric vehicles using deep reinforcement learning with transformer,” *Applied Energy*, vol. 350, p. 121711, 2023. <https://doi.org/10.1016/j.apenergy.2023.121711>
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983. <https://doi.org/10.1126/science.220.4598.671>
- [5] M. Dorigo, V. Maniezzo, and A. Coloni, “The Ant System: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics – Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [6] S. Lin and B. W. Kernighan, “An Effective Heuristic Algorithm for the Traveling-Salesman Problem,” *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973.
- [7] E. H. L. Aarts and J. K. Lenstra (Eds.), *Local Search in Combinatorial Optimization*. Wiley-Interscience Series in Discrete Mathematics and Optimization, John Wiley & Sons, 1997.
- [8] G. B. Dantzig and J. H. Ramser, “The Truck Dispatching Problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959. <https://doi.org/10.1287/mnsc.6.1.80>
- [9] T. Erdelić and T. Carić, “A Survey on the Electric Vehicle Routing Problem: Variants and Solution Approaches,” *Journal of Advanced Transportation*, vol. 2019, Article ID 5075671, 2019. <https://doi.org/10.1155/2019/5075671>
- [10] I. Küçükoğlu, R. Dewil, and D. Cattrysse, “The electric vehicle routing problem and its variations: A literature review,” *Computers & Industrial Engineering*, vol. 161, p. 107650, 2021.
- [11] Google OR-Tools: Operations Research Tools. <https://developers.google.com/optimization>
- [12] Project OSRM – Open Source Routing Machine. <http://project-osrm.org/>
- [13] CVRPLIB: Capacitated Vehicle Routing Problem Library. <http://vrp.atd-lab.inf.puc-rio.br/>

- [14] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.
- [15] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959. <https://doi.org/10.1007/BF01386390>
- [16] G. Boeing, “OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks,” *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017. <https://doi.org/10.1016/j.compenvurbsys.2017.05.004>
- [17] D. Luxen and C. Vetter, “Real-time routing with OpenStreetMap data,” in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 513–516, 2011. <https://doi.org/10.1145/2093973.2094062>
- [18] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proceedings of the 7th Python in Science Conference (SciPy)*, pp. 11–15, 2008.